

1

2

3

4

COMP5621 Computer Networks

Network fundamentals and application protocols

Switching, queueing, HTTP and adaptive video

Lin Fengyan

Problem 1

Zhang Zhixiang

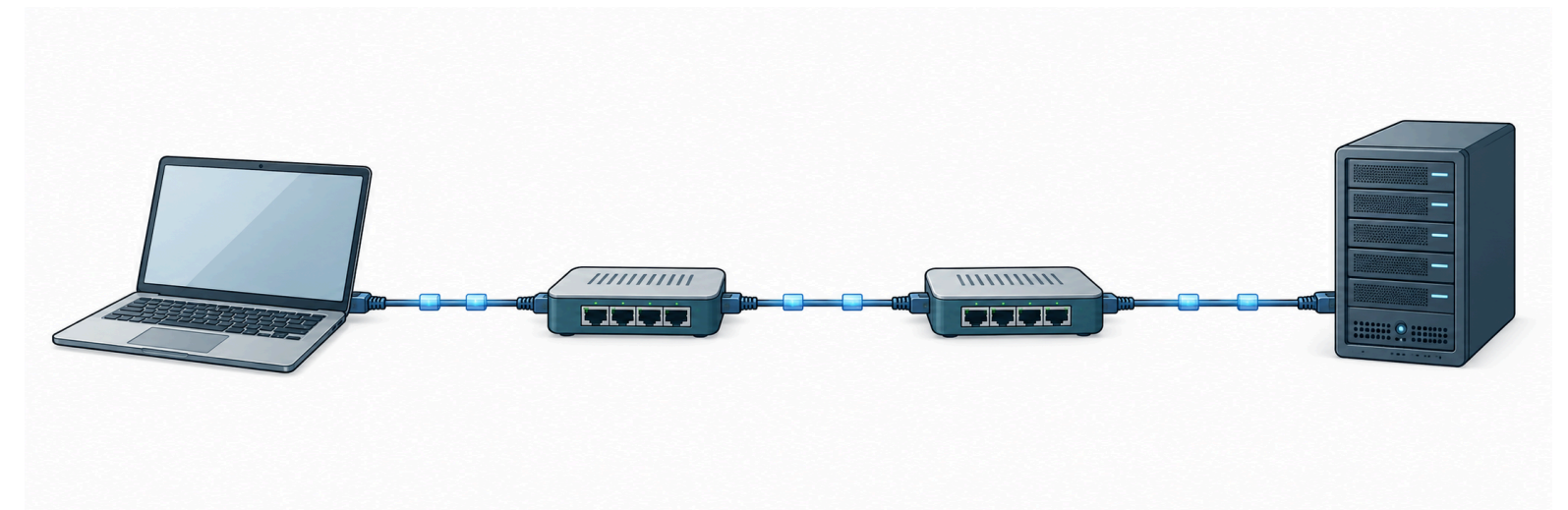
3(a)–3(d)

Gao Yuanyuan

Problem 2

Wang He

3(e) and Problem 4



Problem 1. Circuit, Packet, and Message Switching (30 points)

(a) Circuit switching versus packet switching [6 points]

A 10 Mbps bottleneck link serves users who each require 2 Mbps while active. Circuit switching reserves 2 Mbps for every admitted user throughout the session. Determine the maximum number of users that can be admitted with circuit switching.

1  **Five dedicated reservations fill the link**

	U1	2 Mbps reserved
	U2	2 Mbps reserved
	U3	2 Mbps reserved
	U4	2 Mbps reserved
	U5	2 Mbps reserved

2 $10 \text{ Mbps} / (2 \text{ Mbps per user}) = \mathbf{5 \text{ admitted users}}$

The reservation remains assigned even when its user is inactive.

3 **Circuit switching admits at most 5 users.**

Problem 1. Circuit, Packet, and Message Switching (30 points)

(a) Packet switching and statistical sharing

With packet switching, active users share the 10 Mbps link and no bandwidth is reserved for an individual user. The model is bufferless, so offered traffic above 10 Mbps is lost.

Find the largest simultaneous active-user count without overload, explain why more bursty users can be admitted, and state one drawback.



One shared 10 Mbps link

$5 \times 2 = 10 \text{ Mbps}; 6 \times 2 = 12 \text{ Mbps}$

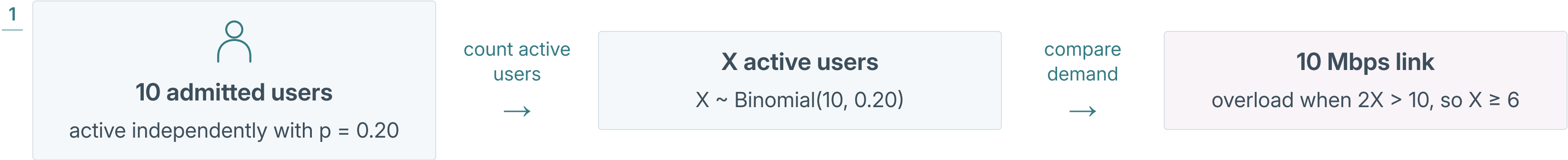
Idle users reserve nothing, so the network may admit more than five bursty users. At most five may be active together without overload.

Its drawback is that simultaneous bursts can overload the link and cause packet loss in this bufferless model; unlike circuit switching, per-user bandwidth and service are not guaranteed

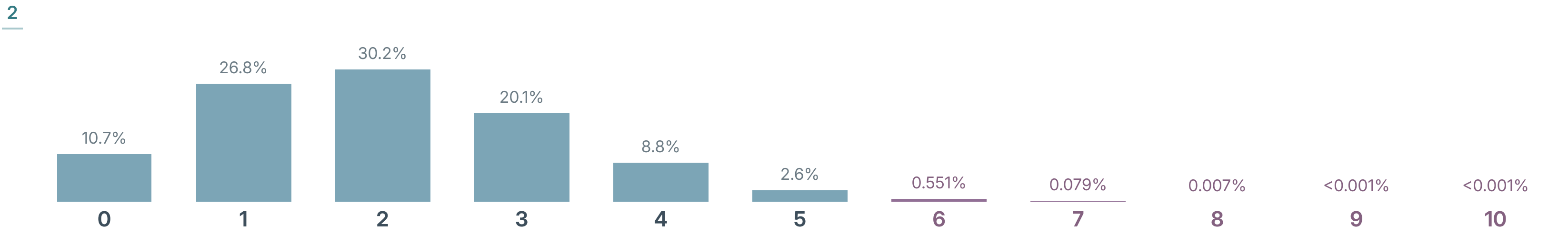
Problem 1. Circuit, Packet, and Message Switching (30 points)

(b) Packet switching: offered load model

Assume $N = 10$ admitted users. Each user is independently active with probability 0.20 and sends at 2 Mbps while active. Compute the mean aggregate offered rate and identify the overload event on a 10 Mbps link.



Mean offered rate = $10 \times 0.20 \times 2 \text{ Mbps} = 4 \text{ Mbps}$



Distribution of active users. Violet bars are overload states.

The 4 Mbps mean does not capture the probability of six or more simultaneous senders.

Problem 1. Circuit, Packet, and Message Switching (30 points)

(b) Exact overload probability and admission

The overload probability must be at most 1%. Determine whether an 11th user can be admitted on the same 10 Mbps link and find the largest admissible N.

1 N = 10

$$P(X \geq 6) = \sum_{k=6}^{10} \binom{10}{k} 0.2^k 0.8^{10-k} = 0.0063693824 \approx 0.6369\%$$

2 N = 11

$$P(X \geq 6) = \sum_{k=6}^{11} \binom{11}{k} 0.2^k 0.8^{11-k} = 0.01165420544 \approx 1.1654\%$$

Users N	Overload on 10 Mbps	1% requirement
10	0.6369%	Pass
11	1.1654%	Fail

4 The 11th user fails the requirement. The largest admissible N is 10.

Problem 1. Circuit, Packet, and Message Switching (30 points)

(b) Minimum capacity for 11 users

If all 11 users must be supported, find the minimum link capacity that satisfies the 1% overload requirement.

1

(a)

(b)

(c)

2

3

4

1 Below 12 Mbps Six active users still overload the link.

At 12 Mbps Six active users fit, so overload starts at seven.

The threshold changes only after another full 2 Mbps user demand fits.

3 Minimum capacity for 11 users = 12 Mbps.

2 11 users, 12 Mbps: overload when $X \geq 7$

$$P(X \geq 7) = \sum_{k=7}^{11} \binom{11}{k} 0.2^k 0.8^{11-k} = 0.0019653632 \approx 0.1965\%$$

0.1965% is below the 1% limit.

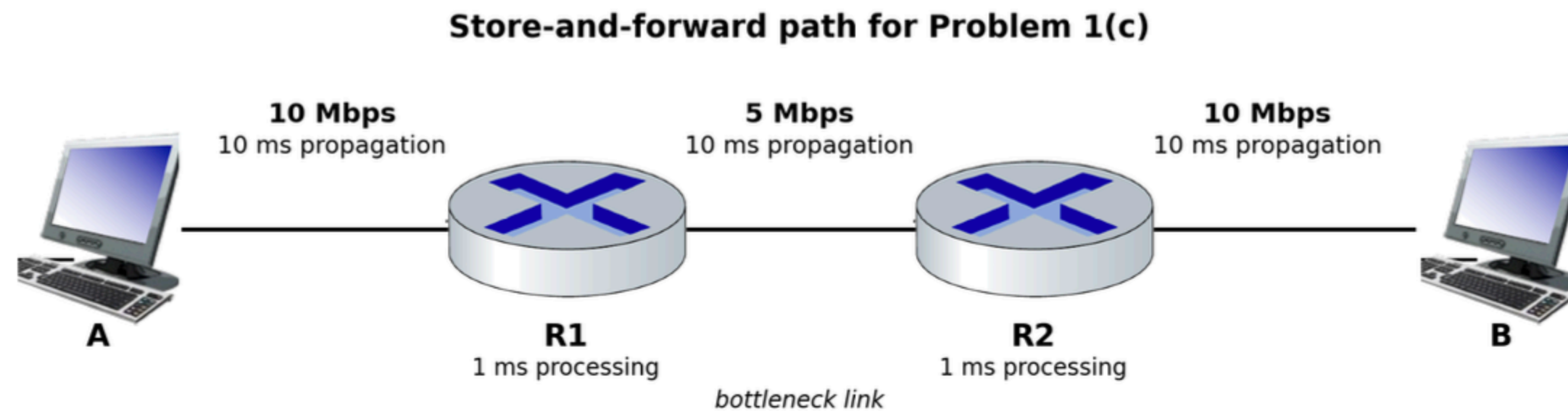
Problem 1. Circuit, Packet, and Message Switching (30 points)

07 / 24

(c) Message switching versus packet switching [12 points]

A sends a 6 Mbit application payload to B through store-and-forward routers R1 and R2. Each router processes a received unit for 1 ms, and every link has a 10 ms propagation delay.

Compare one 6.01 Mbit message with six 1.01 Mbit packets on the 10, 5, and 10 Mbps links.



2 Message switching

6 Mbit payload + 10 Kbit header = 6.01 Mbit

A router waits for the complete message before forwarding it.

3 Packet switching

P1 P2 P3 P4 P5 P6

Each packet carries a 1 Mbit payload and its own 10 Kbit header.

4 The 5 Mbps middle link is the bottleneck for both approaches.

Problem 1. Circuit, Packet, and Message Switching (30 points)

(c) Completion time and pipelining

Compute the message-switching completion time and explain why packet switching can finish earlier even though it sends more header bits.

1 One complete message

The three transmissions cannot overlap because each router must receive all 6.01 Mbit first.

$$\begin{aligned} T &= 0.601 + 1.202 + 0.601 \text{ s} \\ &+ 3 \times 0.010 \text{ s} + 2 \times 0.001 \text{ s} \\ &= \mathbf{2.436 \text{ s}} \end{aligned}$$

2 Six packets in a pipeline

Per-packet transmission times on the three links:

101 ms / 202 ms / 101 ms

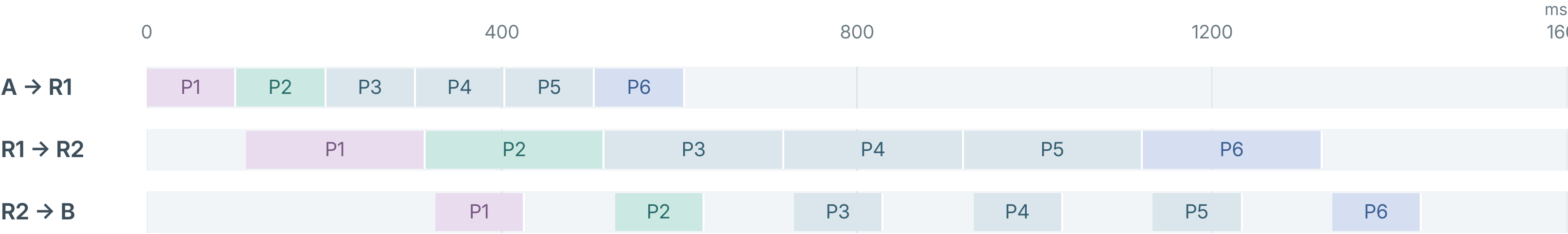
Once P1 reaches R1, R1 forwards it while A sends later packets.
Work overlaps across links.

3 Pipelining outweighs the repeated headers, but smaller packets add header and per-packet processing overhead.

Problem 1. Circuit, Packet, and Message Switching (30 points)

(c) Packet timing on the bottleneck path

Show the timing of P1, P2, and P6, then compute when the last packet reaches B.



Packet	A sends	Ready R1	R1 sends	Ready R2	R2 sends	B receives
P1	0–101	112	112–314	325	325–426	436
P2	101–202	213	314–516	527	527–628	638
P6	505–606	617	1122–1324	1335	1335–1436	1446

Last packet: $436\text{ ms} + 5 \times 202\text{ ms} = 1446\text{ ms} = 1.446\text{ s}$.

Problem 2. Packet Delay and Queueing (20 points)

(a) Delay at a busy output link [8 points]

All packets in this problem are 1,500 bytes long. Router R forwards packets to next hop B over a 6 Mbps FIFO output link. The link is 900 km long, the propagation speed is 2×10^8 m/s, and R takes 0.20 ms to process an arriving packet before it joins the output queue.

Packet P arrives at R and begins its 0.20 ms processing. When processing finishes, the output link is already transmitting a packet with 4,500 bits remaining, and three full packets are waiting ahead of P. Assume that no new arrivals affect P before it is transmitted. Compute P's queueing delay, transmission delay, propagation delay on the R-B link, and the total time from P's arrival at R until B receives its last bit. State the factors that determine transmission delay and propagation delay.



At the router: process P (0.20 ms), then wait for earlier traffic to clear the output.

Only then: transmit P itself. FIFO means first-in, first-out.

Illustrated packets are symbolic; the exact queue is shown below.

P

Target, not yet in service

Q3

Q2

Q1

Three complete packets ahead

4500 bits left

Earlier packet already in service



Output

6 Mbps

3 ⌚ Wait for earlier traffic

$$L = 1500 \text{ bytes} \times 8 = 12,000 \text{ bits}$$

$$\text{Queueing} = (4500 + 3 \times 12,000) \text{ bits} / 6 \text{ Mbps}$$

$$= 0.75 \text{ ms} + 6.00 \text{ ms} = \mathbf{6.75 \text{ ms}}$$

4500 bits are the remaining part of a packet, not another whole 1500-byte packet.

4 📄 Then send P

$$\text{Transmission} = 12,000 \text{ bits} / 6 \text{ Mbps}$$

$$= \mathbf{2.00 \text{ ms}}$$

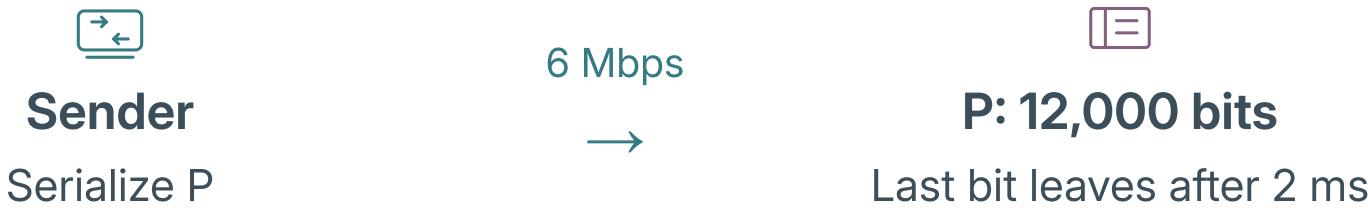
P's own 2 ms transmission is separate from queueing. FIFO prevents P from overtaking the packets already ahead.

Problem 2. Packet Delay and Queueing (20 points)

(a) Delay at a busy output link [8 points]

Packet P arrives at R and begins its 0.20 ms processing. When processing finishes, the output link is already transmitting a packet with 4,500 bits remaining, and three full packets are waiting ahead of P. Assume that no new arrivals affect P before it is transmitted. Compute P's queueing delay, transmission delay, propagation delay on the R-B link, and the total time from P's arrival at R until B receives its last bit. State the factors that determine transmission delay and propagation delay.

1 **Transmission: push bits onto the link**



$L/R = 12,000 \text{ bits} / 6 \text{ Mbps} = \mathbf{2.00 \text{ ms}}$

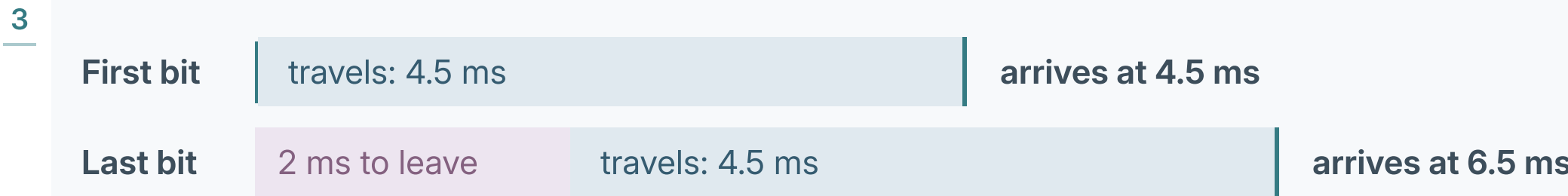
A longer packet takes longer to send. A faster link sends the same packet sooner.

2 **Propagation: travel through the medium**



$d/s = 900,000 \text{ m} / (2 \times 10^8 \text{ m/s})$
 $= \mathbf{4.50 \text{ ms}}$

Distance and signal speed determine travel time, regardless of packet length.



Times above start when transmission starts. Earlier bits propagate while later bits are still being sent. To receive the entire packet, B must wait for the last bit.

4 **Total: processing 0.20 + queueing 6.75 + transmission 2.00 + propagation 4.50 = 13.45 ms.**

Problem 2. Packet Delay and Queueing (20 points)

(b) Traffic intensity and sustained congestion [6 points]

Suppose packets arrive at the same output queue at a long-run average rate of 350 packets/s. Compute the link service capacity in packets/s and the traffic intensity $\rho = \lambda_a/R$. The arrival rate then increases to 525 packets/s and remains at that rate. Recompute ρ . Explain what the two values of ρ imply for long-run queueing behavior, and state the largest long-run arrival rate for which $\rho < 1$.

1

(a)

(b)

(c)



One packet

1500 bytes = 12,000 bits

divide link
bits/s by
bits/packet



Service μ

$6 \times 10^6 / 12,000 = 500$ packets/s

2



350 packets/s: spare service

$\rho = 350 / 500 = 0.70$

Second 1

350 arrive

Up to 500 served

No cumulative deficit

Second 2

350 arrive

Up to 500 served

No cumulative deficit

Second 3

350 arrive

Up to 500 served

No cumulative deficit

150 packets/s of average spare service can clear temporary bursts. This does not mean every packet avoids waiting.

3



525 packets/s: growing backlog

$\rho = 525 / 500 = 1.05$

Second 1

525 arrive

Up to 500 served

25 remain

Second 2

525 arrive

Up to 500 served

50 remain

Second 3

525 arrive

Up to 500 served

75 remain

25 packets/s accumulate. An infinite buffer grows without bound; a finite one eventually drops traffic.

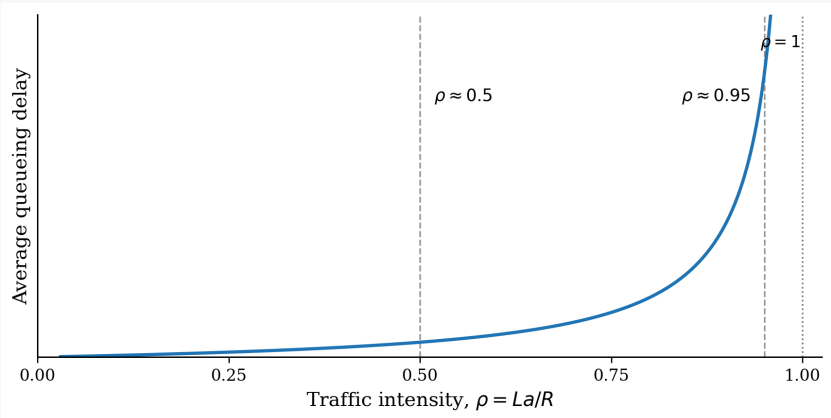
4 The time steps illustrate sustained-rate accounting from no initial backlog; actual bursty arrivals need not occur evenly.

$\rho < 1$ requires $\lambda_a < 500$ packets/s. No attained real-valued maximum; if integer rates are required, 499 packets/s.

Problem 2. Packet Delay and Queueing (20 points)

(c) Traffic intensity and queueing delay [6 points]

The plot below illustrates average queueing delay as the traffic intensity $\rho = \lambda a / R$ increases. (i) Compare the operating conditions around $\rho \approx 0.5$ and $\rho \approx 0.95$. Why does queueing delay become much more sensitive to additional load as ρ approaches 1? (ii) If packet length L and link rate R remain unchanged, by what factor must the average packet arrival rate λ increase to move the system from $\rho = 0.5$ to $\rho = 0.95$? (iii) Based on the plot, explain why operating a link continuously very close to $\rho = 1$ is undesirable even though the long-run offered rate has not yet exceeded the link capacity.



1 At $\rho \approx 0.50$

50% steady traffic

50% spare capacity is available to drain an extra burst.

 **extra burst**  **Faster draining**

Half the service capacity remains after ongoing traffic. Accumulated packets clear relatively quickly.

2 At $\rho \approx 0.95$

95% steady traffic

5% spare capacity is available to drain an extra burst.

 **same burst**  **Slower draining**

Ongoing traffic leaves only 5% spare. A burst persists longer, increasing delay and making delay less predictable.

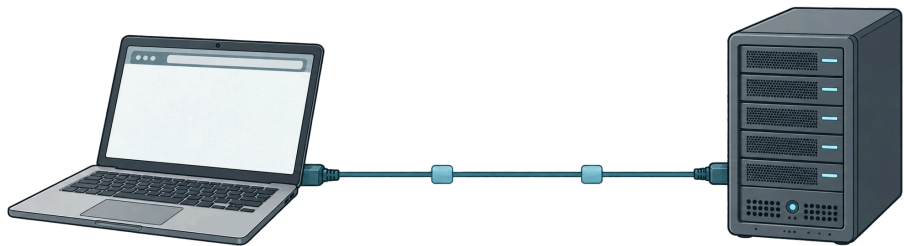
3 With L and R unchanged: $a_{\text{new}} / a_{\text{old}} = 0.95 / 0.50 = \mathbf{1.9}$, a 90% arrival-rate increase.



Problem 3. HTTP and Related Application-Layer Protocols (40 points)

(a) Non-persistent and persistent HTTP [8 points]

A Web page contains one base HTML object and four referenced objects, all on the same server. The round-trip time (RTT) between the browser and server is 50 ms. The base HTML takes 10 ms to transmit, and each referenced object takes 5 ms. Ignore DNS, processing, and queueing delays, and assume that the browser does not open parallel TCP connections. With non-persistent HTTP, the five objects are fetched one at a time, each over a new TCP connection. With persistent HTTP, one TCP connection is reused. After receiving the base HTML, the browser sends the four GET requests back-to-back, and the server returns the four objects back-to-back. For both cases, compute the total time from the start of the first TCP connection setup until all five objects are received. Explain where persistent HTTP saves time and why this advantage grows when a page contains many small objects.



Browser

Server

Browser → server: request

Server → browser: reply

One round trip (RTT) = 50 ms

RTT measures travel to the server and back, not one-way travel. In this problem, a new TCP connection first costs one setup RTT; the GET and start of its response cost another. Object transmission is added afterwards.

2 Read downward: each fetch starts after the previous object finishes.

1. Base
HTML

TCP setup
50 ms

GET / response
50 ms

Transfer
10 ms



2. Object
O1

TCP setup
50 ms

GET / response
50 ms

Transfer
5 ms



3. Object
O2

TCP setup
50 ms

GET / response
50 ms

Transfer
5 ms



4. Object
O3

TCP setup
50 ms

GET / response
50 ms

Transfer
5 ms



5. Object
O4

TCP setup
50 ms

GET / response
50 ms

Transfer
5 ms

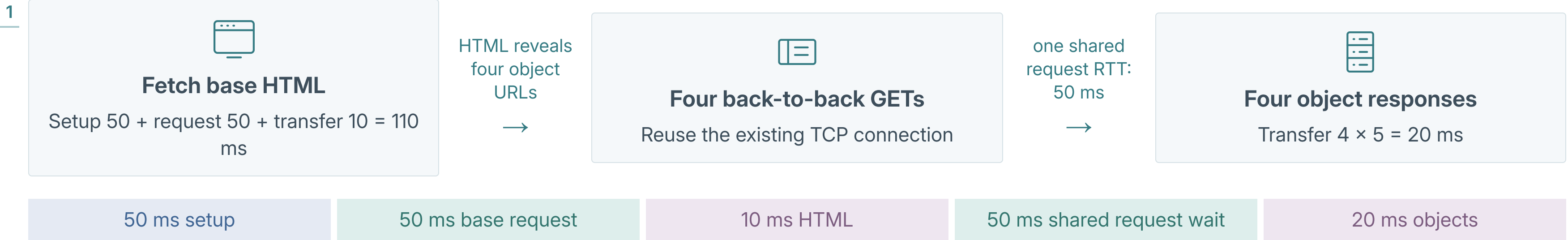
3 Five serial fetches: $5 \times (2 \times 50 \text{ ms}) + 10 \text{ ms} + 4 \times 5 \text{ ms} = 530 \text{ ms}$

Non-persistent HTTP opens a new connection for every object. No parallel connections are allowed here, so none of these five waits can be hidden behind another fetch.

Problem 3. HTTP and Related Application-Layer Protocols (40 points)

(a) Non-persistent and persistent HTTP [8 points]

A Web page contains one base HTML object and four referenced objects, all on the same server. The round-trip time (RTT) between the browser and server is 50 ms. The base HTML takes 10 ms to transmit, and each referenced object takes 5 ms. Ignore DNS, processing, and queueing delays, and assume that the browser does not open parallel TCP connections. With non-persistent HTTP, the five objects are fetched one at a time, each over a new TCP connection. With persistent HTTP, one TCP connection is reused. After receiving the base HTML, the browser sends the four GET requests back-to-back, and the server returns the four objects back-to-back. For both cases, compute the total time from the start of the first TCP connection setup until all five objects are received. Explain where persistent HTTP saves time and why this advantage grows when a page contains many small objects.



2 $110 \text{ ms} + 50 \text{ ms} + 20 \text{ ms} = \mathbf{180 \text{ ms}}$

3 Saved connection setups



$4 \times 50 \text{ ms} = \mathbf{200 \text{ ms}}$

One connection replaces five, removing four setups.

4 Saved serial request waits



$3 \times 50 \text{ ms} = \mathbf{150 \text{ ms}}$

Four requests share one wait instead of four separate waits.

5 $530 - 180 = 350 \text{ ms}$ saved. For small objects, RTT waits dominate their short transmission times.

Phase order only, not to scale. The 180 ms assumes back-to-back GETs; reuse alone does not guarantee it.

Problem 3. HTTP and Related Application-Layer Protocols (40 points)

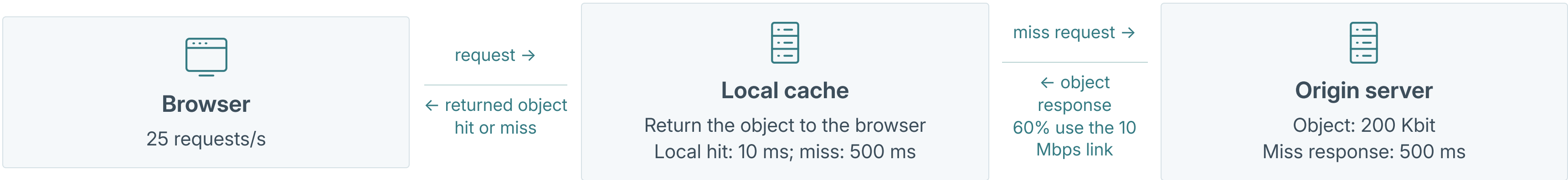
(b) Web caching [8 points]

An institutional network connects to the Internet through a 10 Mbps access link. Its browsers generate an average of 25 Web-object requests per second, and the average object size is 200 Kbits. Assume that the local LAN is not a bottleneck. (i) Without a Web cache, compute the average data rate on the access link and its utilization. (ii) A local Web cache is installed with a hit rate of 0.40. Cache hits are served locally and do not use the access link. Compute the new average access-link data rate and utilization. (iii) If the average response time is 10 ms for a hit and 500 ms for a miss, compute the average response time seen by a user. State two performance benefits of Web caching.

1 4 of 10 representative requests: hits



6 of 10: misses



2 Traffic that crosses the access link

No cache: $25 \times 200 \text{ Kbit} = 5 \text{ Mbps}$

Utilization: $5 / 10 = 50\%$

With cache: $0.60 \times 5 = 3 \text{ Mbps}$

Utilization: $3 / 10 = 30\%$

3 Weight the two response paths

$$0.40 \times 10 \text{ ms} + 0.60 \times 500 \text{ ms}$$

$$= 304 \text{ ms}$$

Hits are 40%, not half the requests. A simple average of 10 and 500 ms would give the wrong weighting.

4 Local hits return faster; fewer misses reduce upstream traffic and congestion.

Ten tokens show the 40/60 proportion, not ten measured requests. Miss time is the given 500 ms; no extra queueing term is added.

Problem 3. HTTP and Related Application-Layer Protocols (40 points)

(c) HTTP/1.1 and HTTP/2: response semantics and object delivery [10 points]

A server returns the following HTTP/1.1 response for the base HTML object of a Web page:

- (i) State the meaning of the status line and the roles of Content-Type, Last-Modified, Set-Cookie, and Connection: keep-alive.
- (ii) If the same Web interaction uses HTTP/2 instead of HTTP/1.1, which application-level semantics above remain essentially unchanged? Explain what changes in the delivery of multiple requested objects.

1

- A HTTP/1.1 200 OK
- B Content-Type: text/html
Content-Length: 12000
- C Last-Modified: Tue, 08 Sep 2026 10:00:00 GMT
- D Set-Cookie: session_id=12345
- E Connection: keep-alive

Blank line, then the HTML body

2

- A Request succeeded; this is the response status.
- B The response body is HTML, not an image or video.
- C Modification time supports later cache validation.
- D Browser stores a session cookie for later matching requests.
- E Keep this HTTP/1.1 TCP connection available for reuse.

3 This is a response, so Set-Cookie asks the browser to store a value. A later matching request sends the cookie back; it is not the HTML content itself.

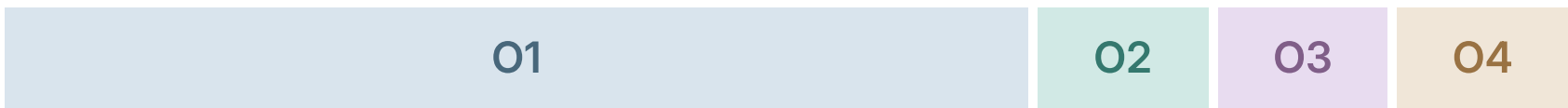
HTTP/2 keeps status, content metadata, cache and cookie meanings. It changes their framing and delivery. Connection: keep-alive is excluded from HTTP/2.

Problem 3. HTTP and Related Application-Layer Protocols (40 points)

(c) HTTP/1.1 and HTTP/2: response semantics and object delivery [10 points]

- (iii) The browser requests one large object O1 followed by three small objects O2, O3, and O4 over one connection. With HTTP/1.1, assume that the requests are pipelined but the server sends complete responses in first-come-first-served (FCFS) order. Explain why O2-O4 may wait behind O1. Then explain how HTTP/2 framing, interleaving, and client-specified priority can reduce this waiting.
- (iv) Suppose a TCP segment carrying part of O1 is lost. Does HTTP/2 completely remove head-of-line blocking? Explain what can still happen to the other streams that share the same TCP connection.

1 HTTP/1.1 pipelined FCFS



Complete O1 first, then O2–O4. The small objects cannot overtake the large response under the stated ordering.

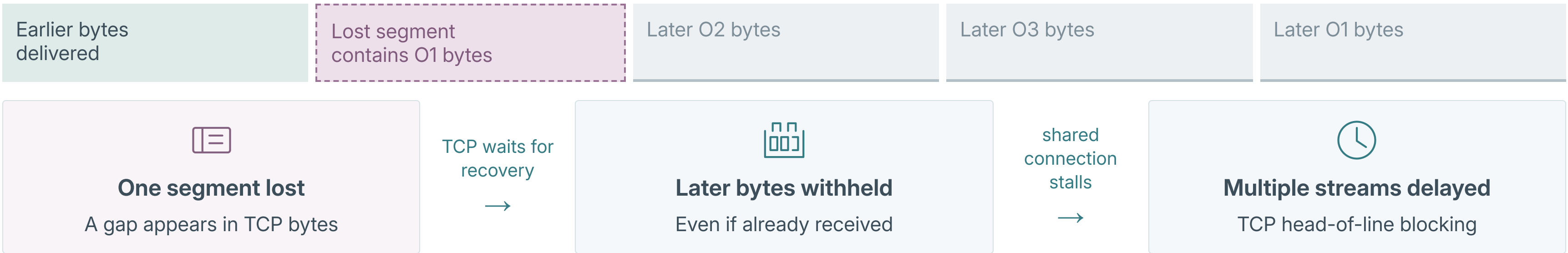
Object colors match across both strips. The strips illustrate ordering, not measured transmission time.

2 HTTP/2 multiplexed streams



Split responses into frames with stream IDs. Interleaving lets small objects finish before O1; client priority can guide server scheduling.

3 All those streams still share one ordered TCP byte sequence

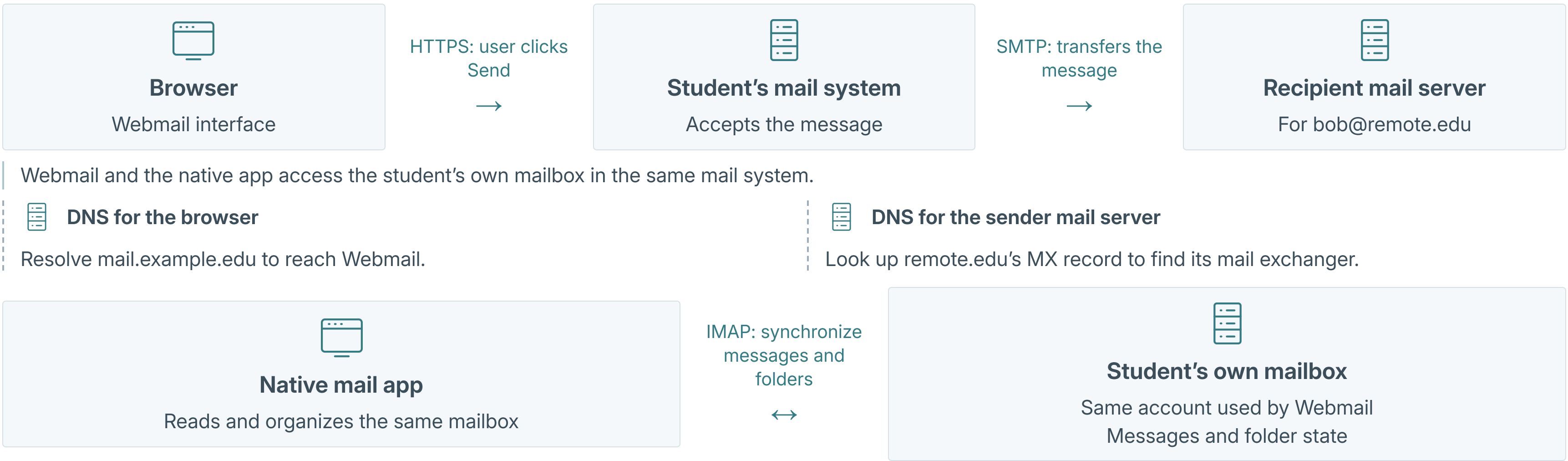


4 **HTTP/2 removes whole-response serialization, but does not fully remove TCP-level head-of-line blocking.**

Problem 3. HTTP and Related Application-Layer Protocols (40 points)

(d) DNS and email [6 points]

A student visits `https://mail.example.edu` in a browser, sends a message to `bob@remote.edu`, and later reads and organizes the same mailbox using a native mail application. Explain the roles of DNS, HTTP, SMTP, and IMAP in this scenario. Which DNS resource-record type helps the sender's mail server locate the mail server for `remote.edu`? State one benefit and one possible drawback of DNS caching.



3 DNS caching: reuse answers until expiry

First lookup

Query DNS; cache answer

Within TTL

Reuse locally: faster, fewer queries

Record changes before TTL expires

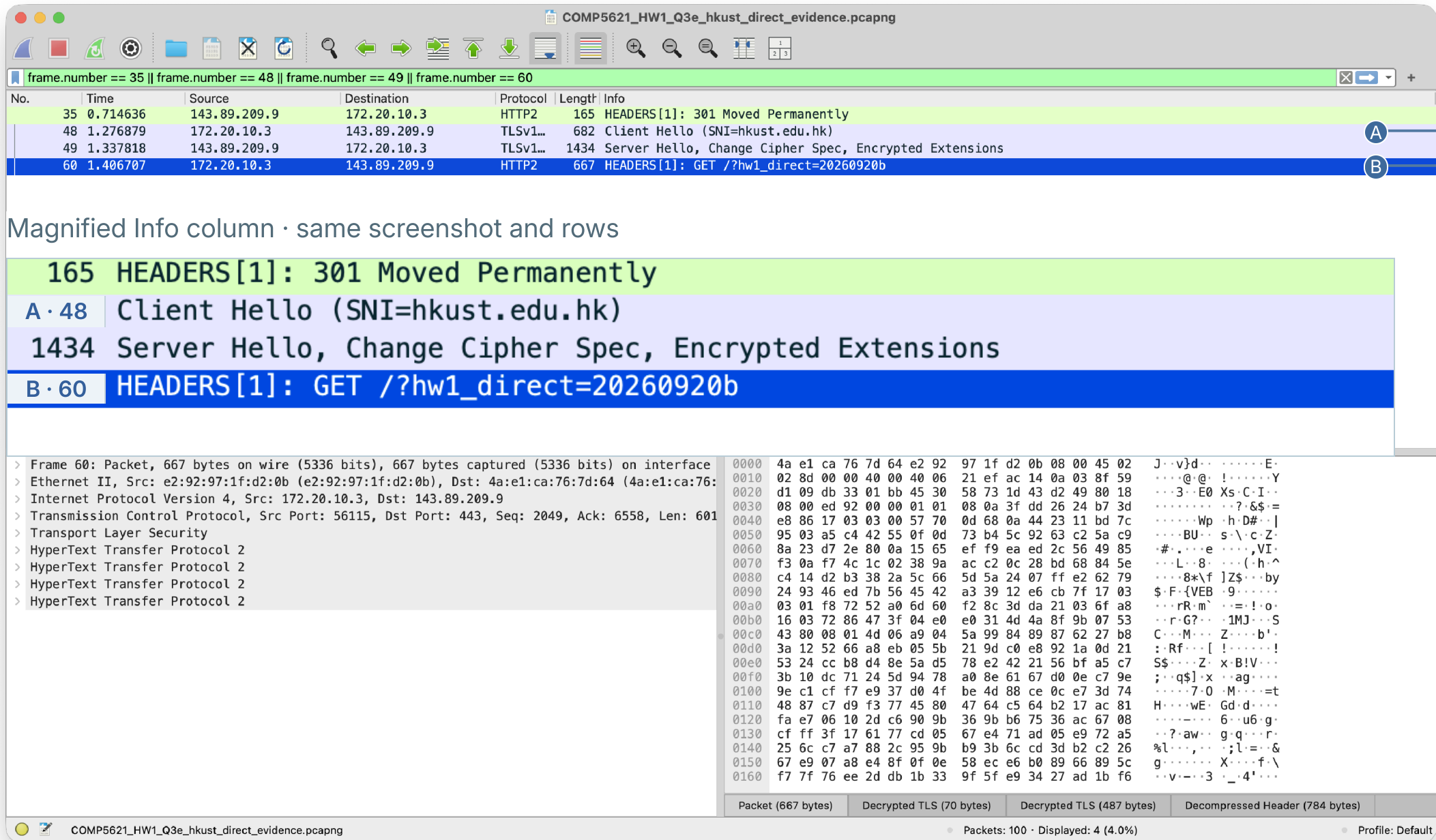
Cached old answer can remain stale

The browser does not directly deliver SMTP mail to the recipient. DNS locates services; it does not carry the email body.

Problem 3. HTTP and Related Application-Layer Protocols (40 points)

(e) Wireshark: HTTP version [8 points]

Using Wireshark, capture traffic while visiting www.ust.hk and www.cse.ust.hk. Determine the HTTP version used to retrieve the main page of each site. For each site, include one screenshot and state the evidence in the capture that supports your answer. If the site redirects, identify the final hostname and examine the connection that serves the main page. Report the version observed in your capture; the two sites may use different HTTP versions.



A Frame 48: Final hostname

This Client Hello row says **SNI=hkust.edu.hk**. It identifies the hostname used by this connection.

B Frame 60: Actual page request

The selected row is **HTTP2 HEADERS[1]: GET /?hw1_direct=20260920b**. Wireshark decoded the actual request as HTTP/2.

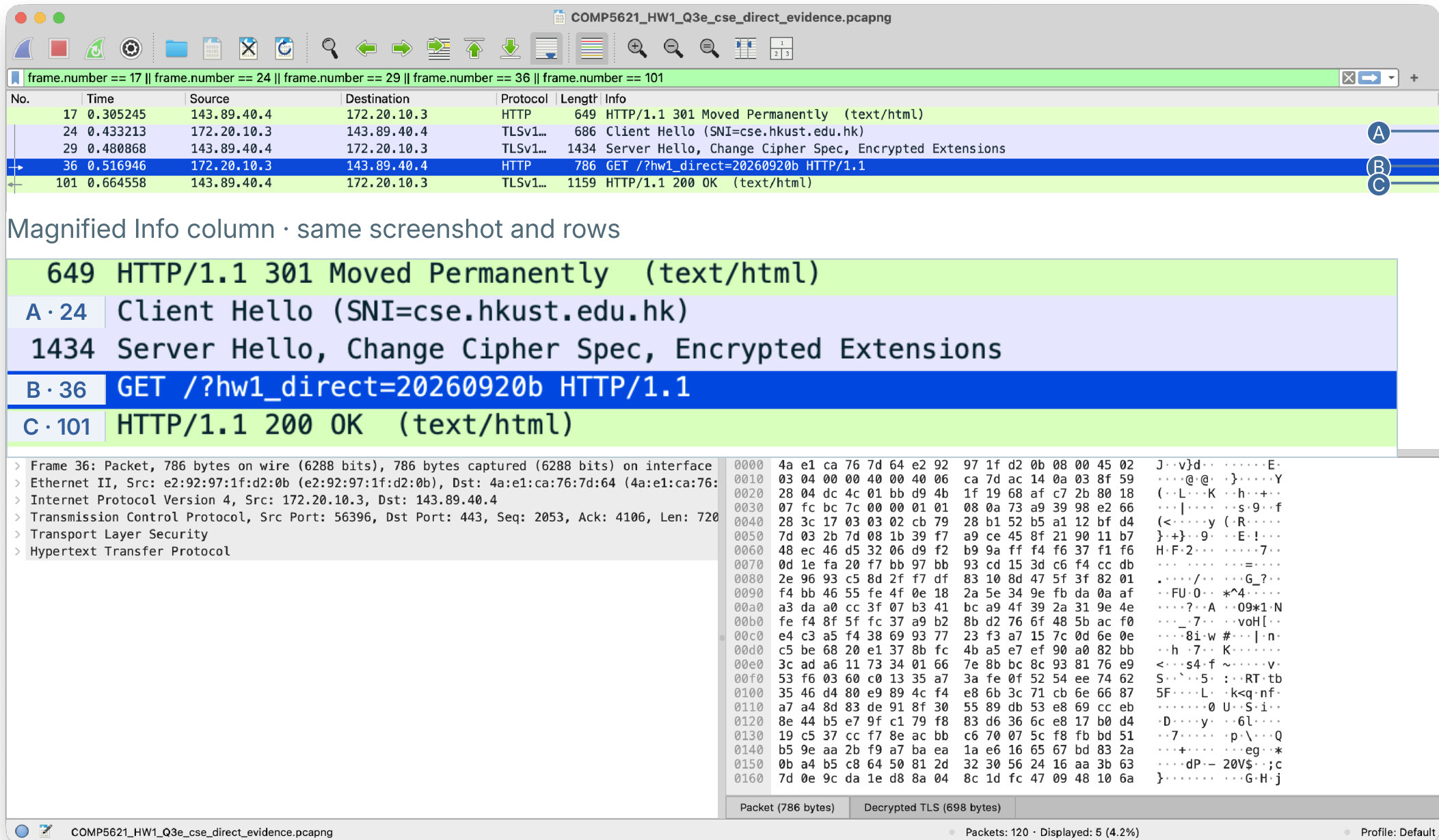
Client 172.20.10.3 → server 143.89.209.9. Full original screenshot; click to inspect.

www.ust.hk redirects to hkust.edu.hk. A identifies the final host; B establishes observed HTTP/2 on the page request.

Problem 3. HTTP and Related Application-Layer Protocols (40 points)

(e) Wireshark: HTTP version [8 points]

Using Wireshark, capture traffic while visiting www.ust.hk and www.cse.ust.hk. Determine the HTTP version used to retrieve the main page of each site. For each site, include one screenshot and state the evidence in the capture that supports your answer. If the site redirects, identify the final hostname and examine the connection that serves the main page. Report the version observed in your capture; the two sites may use different HTTP versions.



A Frame 24: Final hostname

SNI in this row is **cse.hkust.edu.hk**.

B Frame 36: Actual request version

The selected request explicitly reads **GET /?hw1_direct=20260920b HTTP/1.1**.

C Frame 101: HTML response

Info reports **HTTP/1.1 200 OK (text/html)**. This is the successful HTML response, not just a handshake.

Client 172.20.10.3 → server 143.89.40.4. Protocol candidates in Client Hello alone do not prove the selected version.

www.cse.ust.hk redirects to cse.hkust.edu.hk. A names the host; B proves HTTP/1.1; C supports the page-response interpretation.

Problem 4. Adaptive Video Streaming with DASH (10 points)

(a) Adaptive rate selection [4 points]

A DASH server stores each 4-second video chunk at three encoding rates: 1 Mbps, 2 Mbps, and 4 Mbps. Before each download, the client estimates the sustainable server-to-client bandwidth. For three consecutive chunks, the estimates are 5 Mbps, 3 Mbps, and 1.5 Mbps. Assume the client chooses the highest available encoding rate that does not exceed the estimate, the actual download throughput equals the estimate, and protocol overhead is negligible.

For each of the three chunks, choose the encoding rate and compute the chunk download time.

1 Encoding rate measures video data per second of playback. Throughput measures data delivered per second of downloading. First select an available encoding, then compute size and download time.

2 Chunk 1

↔↔↔ **5 Mbps throughput**

Available: 1 2 **4** Mbps

Choose highest ≤ 5 Mbps

▶ **4 Mbps encoding**

Multiply by 4 s of video

$4 \text{ Mbps} \times 4 \text{ s} = \mathbf{16 \text{ Mbit}}$

Divide by network throughput

$16 \text{ Mbit} / 5 \text{ Mbps}$
 $= \mathbf{3.20 \text{ s}}$

3 Chunk 2

↔↔↔ **3 Mbps throughput**

Available: 1 **2** 4 Mbps

Choose highest ≤ 3 Mbps

▶ **2 Mbps encoding**

Multiply by 4 s of video

$2 \text{ Mbps} \times 4 \text{ s} = \mathbf{8 \text{ Mbit}}$

Divide by network throughput

$8 \text{ Mbit} / 3 \text{ Mbps}$
 $= \mathbf{2.67 \text{ s}}$

4 Chunk 3

↔↔↔ **1.5 Mbps throughput**

Available: **1** 2 4 Mbps

Choose highest ≤ 1.5 Mbps

▶ **1 Mbps encoding**

Multiply by 4 s of video

$1 \text{ Mbps} \times 4 \text{ s} = \mathbf{4 \text{ Mbit}}$

Divide by network throughput

$4 \text{ Mbit} / 1.5 \text{ Mbps}$
 $= \mathbf{2.67 \text{ s}}$

5 **All downloads finish within 4 s of playback. At 3 Mbps choose 2, not 3: only 1, 2 and 4 Mbps encodings exist.**

Problem 4. Adaptive Video Streaming with DASH (10 points)

(b) Why rate adaptation matters [3 points]

Suppose the client always selects the 4 Mbps representation. Compute the download time during the 3 Mbps and 1.5 Mbps bandwidth periods. Explain why repeated downloads that take longer than the 4-second playback duration can cause playback stalls and why DASH changes the encoding rate.

Keeping 4 Mbps encoding fixes every 4 s chunk at 16 Mbit. Playback continues while a chunk downloads; the buffer must cover the difference.

1 3 Mbps network

$$16 \text{ Mbit} / 3 \text{ Mbps} \approx \mathbf{5.33 \text{ s}}$$

 **5.33 s**

playback consumes during download

 **4.00 s**

new playable video arrives

$$\text{Buffer change} = 4 - 5.33 = \mathbf{-1.33 \text{ s/chunk}}$$

2 1.5 Mbps network

$$16 \text{ Mbit} / 1.5 \text{ Mbps} \approx \mathbf{10.67 \text{ s}}$$

 **10.67 s**

playback consumes during download

 **4.00 s**

new playable video arrives

$$\text{Buffer change} = 4 - 10.67 = \mathbf{-6.67 \text{ s/chunk}}$$

3 Repeated slow chunks

B_0

$B_0 - \Delta$

$B_0 - 2\Delta$

$B_0 - 3\Delta$

Eventually no
playable reserve

B_0 is an unspecified initial reserve. $\Delta = 1.33 \text{ s}$ or 6.67 s here, assuming uninterrupted playback and sequential downloads. Once reserve is exhausted, playback stalls; one slow chunk need not stall immediately.

4



Falling reserve

Downloads keep taking too long

adapt before
exhaustion



Lower encoding

Fewer bits in the next chunk

same network,
smaller
transfer



Shorter download

Helps keep pace with playback

Problem 4. Adaptive Video Streaming with DASH (10 points)

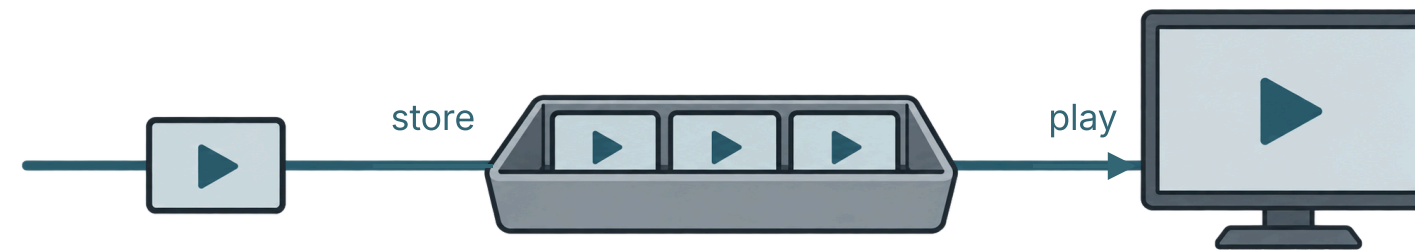
(c) Buffering and CDN [3 points]

Explain how a client-side playout buffer helps when network delay or throughput varies. State one reason why a nearby CDN node can improve streaming performance.

Download
Network delivers chunks

Buffer
Already downloaded video

Playback
Consumes stored video



Storage metaphor: the buffer is memory in the player, not a physical box on the network.

2 Temporary variation

Network briefly slows

Buffer supplies already downloaded chunks

Network recovers and replenishes reserve

The buffer absorbs short-term timing variation. If delivery stays below consumption, the reserve still drains and eventually playback stalls.

3 A nearby CDN may improve the path

Distant origin

longer / possibly congested route



Nearby CDN

shorter / possibly less congested route



A shorter or less congested route can reduce RTT and improve chunk delivery. Proximity is a potential improvement, not a guarantee of capacity.

4 Buffering handles brief variation; CDN placement can improve delivery. Neither cures an indefinitely sustained rate deficit.